REGULAR EXPRESSIONS

Overview of Regular Expressions

- Regular Expressions (regex) are powerful tools for matching patterns in text. In Python, the re module provides a suite of functions for working with regular expressions.

- Regular expressions are a versatile tool in any Python developer's toolkit, enabling efficient and powerful text manipulation.

Key Features

- Text Processing: RE is essential for tasks like searching, replacing, and parsing text.
- Data Validation: Used to validate inputs such as email addresses, phone numbers, and other structured data.
- Data Extraction: Efficiently extract information from large text datasets.

Special Sequences

\d: Matches any digit (equivalent to [0-9]).

\D: Matches any non-digit character.

\w: Matches any alphanumeric character (equivalent to [a-zA-Z0-9_]).

\W: Matches any non-alphanumeric character.

\s: Matches any whitespace character (spaces, tabs, newlines).

\S: Matches any non-whitespace character.

```
# Define patterns for special sequences
pattern_digits = r'\d'  # Matches any digit
pattern_words = r'\w'   # Matches any word character
pattern_whitespace = r'\s'  # Matches any whitespace
# Find all digits
digits = re.findall(pattern_digits, text)
print("Digits found:", digits)
# Find all word characters
words = re.findall(pattern_words, text)
print("Word characters found:", words)
# Find all whitespace characters
whitespace = re.findall(pattern_whitespace, text)
print("Whitespace characters found:", whitespace)
```

Pattern Matching

- Literal Characters: Match the exact characters in the text.
- Metacharacters: Special characters that define search patterns

(e.g., ., ^, $, *, +, ?, {}, [], |, (), \).

```
import re

# Sample text
text = "Please contact us at support@example.com"

# Define the pattern for an email address
email_pattern = r'[A-z0-9._%+-]+@[A-z0-9.-]+\.[A-z]{2,}'

# Find all email addresses in the text
matches = re.findall(email_pattern, text)

# Print the found email addresses
print("Email addresses found:", matches)
```

Groups and Ranges

[]: Matches any single character within the brackets.

(): Groups patterns for applying operators or extracting matched substrings.

```
# Sample text
text = "Sample text with numbers 12345 and mixed CASE letters."
# Define patterns using ranges
pattern_digits = r'[0-9]+'  # Matches sequences of digits
pattern_lowercase = r'[a-z]+'  # Matches sequences of lowercase letters
pattern_uppercase = r'[A-Z]+'  # Matches sequences of uppercase letters
# Find all sequences of digits
digits = re.findall(pattern_digits, text)
print("Digits found:", digits)
# Find all sequences of lowercase letters
lowercase = re.findall(pattern_lowercase, text)
print("Lowercase letters found:", lowercase)
# Find all sequences of uppercase letters
uppercase = re.findall(pattern_uppercase, text)
print("Uppercase letters found:", uppercase)
```

Quantifiers

*: Matches 0 or more repetitions.

+: Matches 1 or more repetitions.

?: Matches 0 or 1 repetition.

{n,m}: Matches n and m repetitions.

```
import re

# Sample text
text = "The price is 100 dollars."

# Define patterns using quantifiers
pattern_digits = r'\d{3}'  # Matches exactly three digits

# Find all patterns matching exactly three digits
three_digits = re.findall(pattern_digits, text)
print("Three digits found:", three_digits)
```

Non-Greedy (Lazy) Quantifiers

- Non-greedy quantifiers match **as little text as possible** while still allowing the pattern to succeed.

- Lazy = “match minimally, expand only if required.”

Example:

Pattern: `<a>.*?</a>`

Text: `<a>one</a> text <a>two</a>`

Match: `<a>one</a>`

Greedy Quantifiers

- Greedy quantifiers match **as much text as possible** while still allowing the overall pattern to succeed.

- Greedy = “consume first, backtrack only if forced.”

Example:

Pattern: <a>.*

Text: <a>one text <a>two

Match: <a>one text <a>two

Lookaheads

- Match **based on what follows**, without consuming characters
- Useful for validation, token boundaries, and conditional matching
- Zero-width: the cursor never moves

Example:

Pattern: r"\w+(?=@example\.com)"

Text: Match only if the next characters satisfy a condition

Match: **only if** followed by @example.com

Negative Lookahead

- Match only if the next characters **do not** satisfy a condition.
- Zero-width: the cursor never moves

Example:

Pattern: r"^(?!.*password).*$"

Text: Match only if the next characters satisfy a condition

Match: Matches any line not containing “password”

Summary

- The re module in Python equips developers with a powerful and flexible tool for handling complex text manipulation tasks.
- Its ability to perform pattern matching, validation, and efficient text processing makes it an important tool in many programming scenarios.

Tutorials

- Regular Expression Tutorial

https://www.geeksforgeeks.org/python-regex/

- Regular Expression Cheat-Sheet

https://cheatography.com/davechild/cheat-sheets/regular-expressions/

- HTML Tutorial

https://www.w3schools.com/html/